produces:

```
About to require file using refinement
Dave says: I'M HERE!
Back from require
Dave says: FINISHED
```

Notice how the puts calls in the main program are unadorned, but the calls in the file that uses the refinement has the vanity prefix.

Designing with Refinements

In general, refinements can be used any place you would want to extend or monkey-patch a third-party tool. There are two main use cases.

The first is the case where a developer wants to make changes to a third-party class for the developer's own use. For example, Rake, which issues a lot of calls to run external programs using the system method, might want to modify the built-in version of system so that it logs errors differently. However, it does not want that logging to apply to other calls to system that are not part of Rake. In this case, the code could use the refinement locally within its own source files. The refinement would be an implementation detail, hidden from users of the code.

The second use case is where a library writer offers the refinement as part of the external interface. For example, the Rails Active Support gem defines methods such as hours, minutes, and ago on numbers, allowing you to write 3.days.ago. Right now, those changes to numbers are global. But, using refinements, the Rails team could code the new methods, but not add them in to any system classes. Instead, their API would document how to add them for yourself into just those source files that uses them. They might tell you to write

```
using Rails::Extensions::Durations
```

in any source file that needs to use them.

Obviously there are many more use cases. And the two we've mentioned are not mutually exclusive. The Rails framework, for example, is likely to want to use these duration-related methods itself, as well as making them available via a documented refinement.

Metaprogramming Class-Level Macros

If you've used Ruby for any time at all, you're likely to have used attr_accessor, the method that defines reader and writer methods for instance variables:

```
class Song
  attr_accessor :duration
end
```

If you've written a Ruby on Rails application, you've probably used has_many:

```
class Album < ActiveRecord::Base
  has_many :tracks
end
```

These are both examples of class-level methods that generate code behind the scenes. Because of the way they expand into something bigger, folks sometimes call these kinds of methods *macros*.